

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM**

MODUL 1, 2 dan 3

MOBILE



Disusun oleh:

VALISHA ATTHALIA NAURA IRFAN

2311102160

S1 IF-11-04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

MODUL 1

Running Modul

DASAR TEORI

1. Running Modul

1.1. Git

Git merupakan sistem pengontrol versi terdistribusi (DVCS) yang dirancang oleh Linus Torvalds untuk mengelola riwayat perubahan dalam pengembangan perangkat lunak secara efisien. Berbeda dengan sistem konvensional yang bersifat terpusat, Git memberikan keleluasaan bagi setiap pengembang untuk memiliki salinan basis data (*repository*) yang utuh di perangkat lokal masing-masing. Pendekatan ini tidak hanya meningkatkan kecepatan akses karena pengerjaan bisa dilakukan secara *offline*, tetapi juga memberikan keamanan tambahan; jika server utama bermasalah, seluruh riwayat proyek masih tersimpan aman di setiap komputer anggota tim.

1.2. Instalasi JDK

Java Development Kit (JDK) merupakan perangkat yang digunakan untuk melakukan proses kompilasi dari kode Java ke *bytecode*. *Bytecode* ini nantinya dapat dimengerti dan dijalankan oleh JRE (*Java Runtime Environment*).

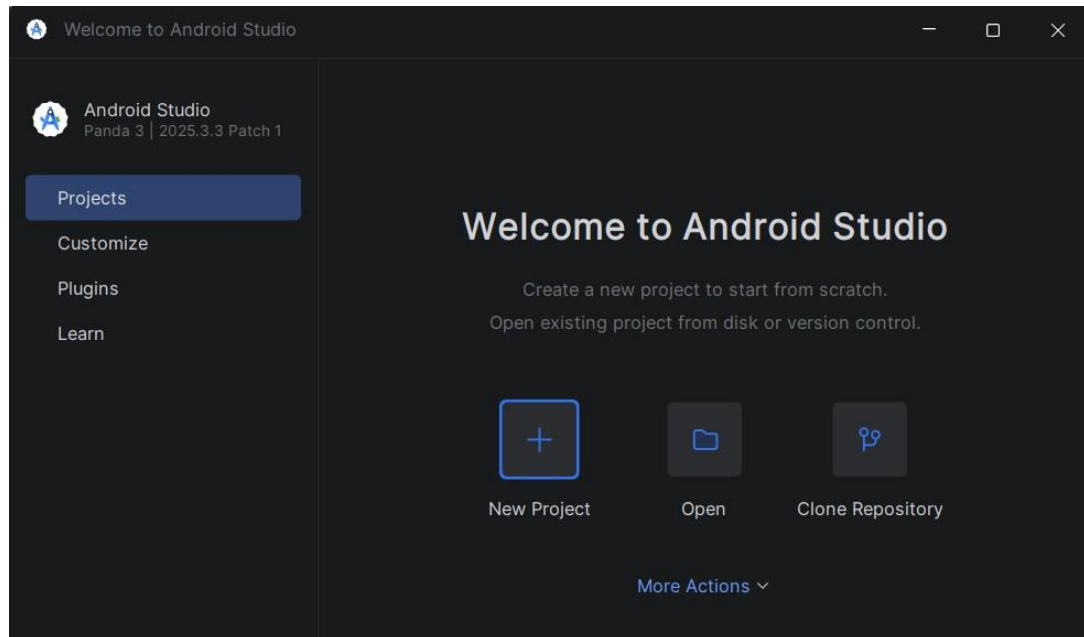
1.3. Instalasi Flutter SDK

Flutter SDK adalah alat pengembangan yang bergantung pada *tools* yang ada pada *environment* sistem operasi pengguna. Pada OS Windows, Flutter bergantung pada *Windows PowerShell* dan *Git*.

```
C:\Users\user>flutter --version
Flutter 3.44.1 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 924134a44c (3 weeks ago) • 2026-05-29 12:13:22 -0400
Engine • hash 39b1f7043775b9578bbb26a1676e79c4e31c8b5e (revision c416acfeb8) (21 days ago) • 2026-05-27 20:19:31.000Z
Tools • Dart 3.12.1 • DevTools 2.57.0
```

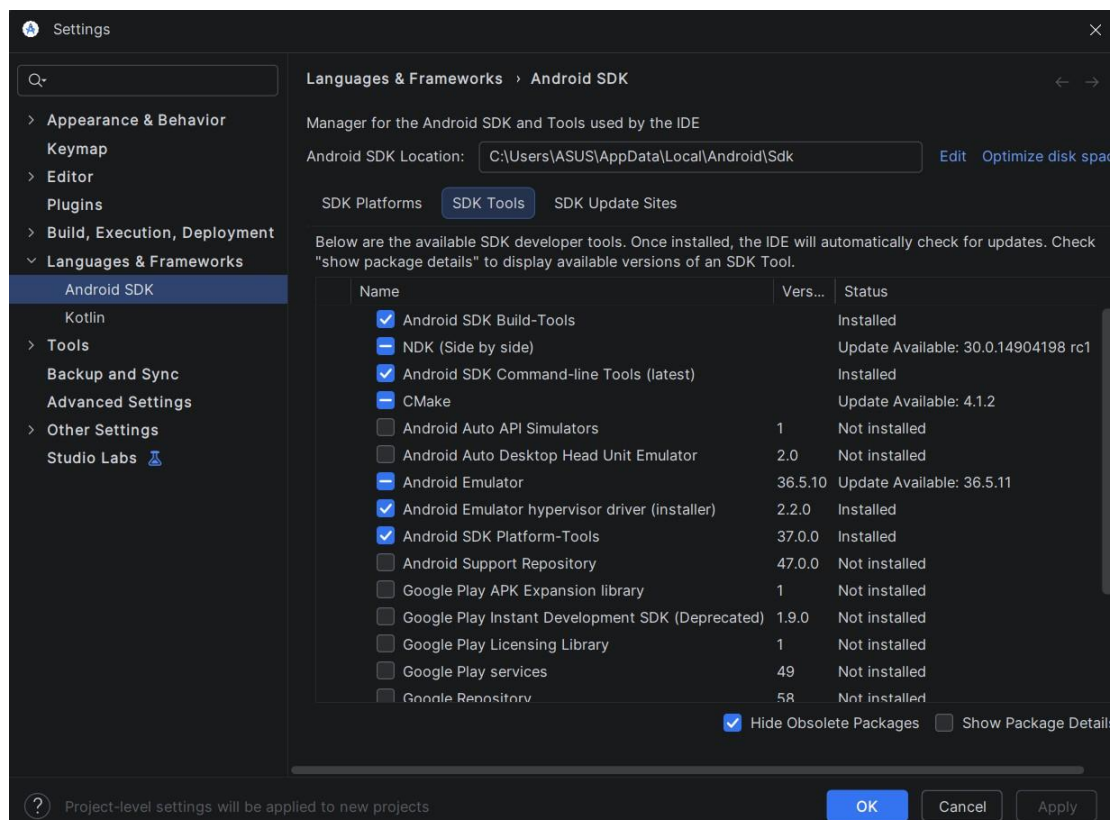
1.4. Instalasi Android Studio

Android Studio merupakan IDE resmi dalam membangun aplikasi berbasis Android.



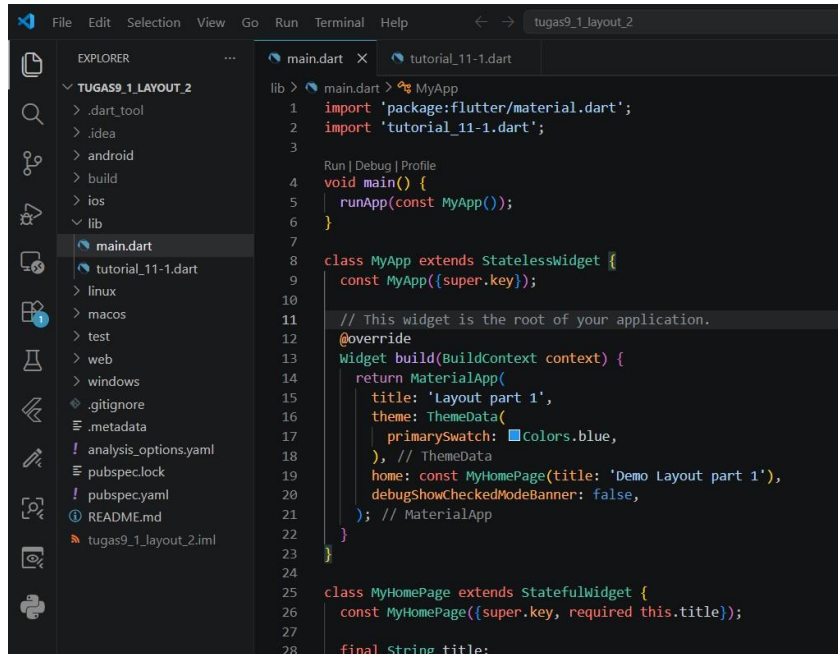
1.5. Instalasi SDK Android

SDK (Standard Development Kit) adalah kumpulan alat, komponen, dan platform yang wajib ada pada Android Studio untuk mengembangkan aplikasi Android.

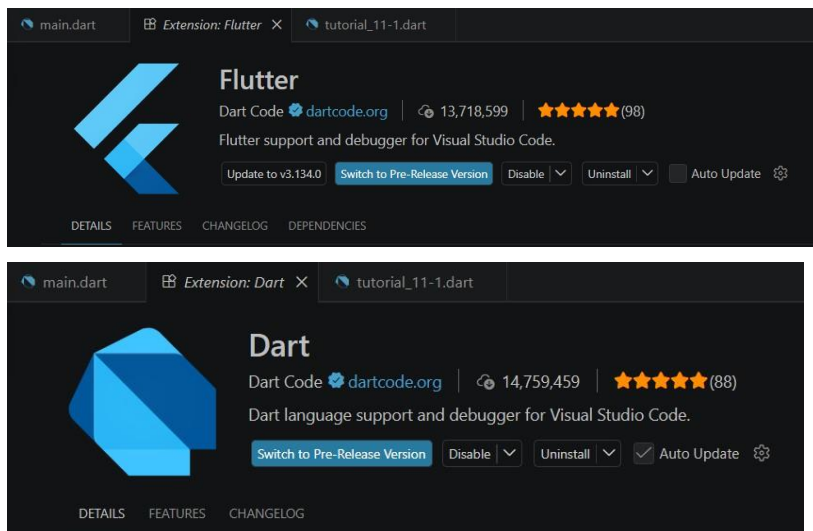


1.6. Instalasi Visual Studio Code

Visual Studio Code (VSCode) merupakan *editor* kode yang dapat digunakan untuk membuat aplikasi Flutter. Editor ini memerlukan instalasi ekstensi Dart dan Flutter tambahan agar aplikasi dapat dibuat dan berjalan tanpa error.



1.6.1. Instalasi Extension Visual Code



MODUL 2

Pengenalan Flutter

DASAR TEORI

2. Flutter

Flutter dibangun menggunakan perpaduan bahasa C, C++, dan Dart, serta memanfaatkan mesin grafis Google's Skia untuk me-render antarmuka pengguna. Karena berjalan di atas *Dart Virtual Machine* (VM) yang didukung oleh kompilasi *just-in-time* (JIT), Flutter memiliki fitur unggulan bernama *hot-reload* yang sangat mempercepat proses pengembangan aplikasi. Di dalam Flutter, setiap elemen pada layar disebut sebagai *widget*. Antarmuka aplikasi disusun menggunakan sistem widget tree, di mana komponen-komponen kecil dirangkai menjadi satu kesatuan tampilan. Secara perilaku, *widget* terbagi dua: *stateless* untuk tampilan yang statis atau tetap, dan *stateful* untuk komponen dinamis yang bisa berubah mengikuti status (data) aplikasi.

2.1. Arsitektur Flutter

Business Logic Component (BLoC) adalah arsitektur yang direkomendasikan untuk mengelola *state* pada Flutter. Tujuan utama arsitektur ini adalah memisahkan kode tampilan (UI) dari logika bisnisnya. Dengan pendekatan ini, penanganan setiap event dan perubahan data menjadi lebih rapi (*simplicity*), lebih mudah dikembangkan (*scalability*), dan mudah untuk diuji (*testability*).

2.2. Hello World Pada Flutter

Pembuatan project baru untuk pengenalan/penggunaan Flutter, sebagai berikut:
contoh kode 'hello world' pada file main.dart, dalam project flutter:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);
  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Hello World",
      home: const MyHomePage(title: "Flutter Hello World Page"),
    );
  }
}
```

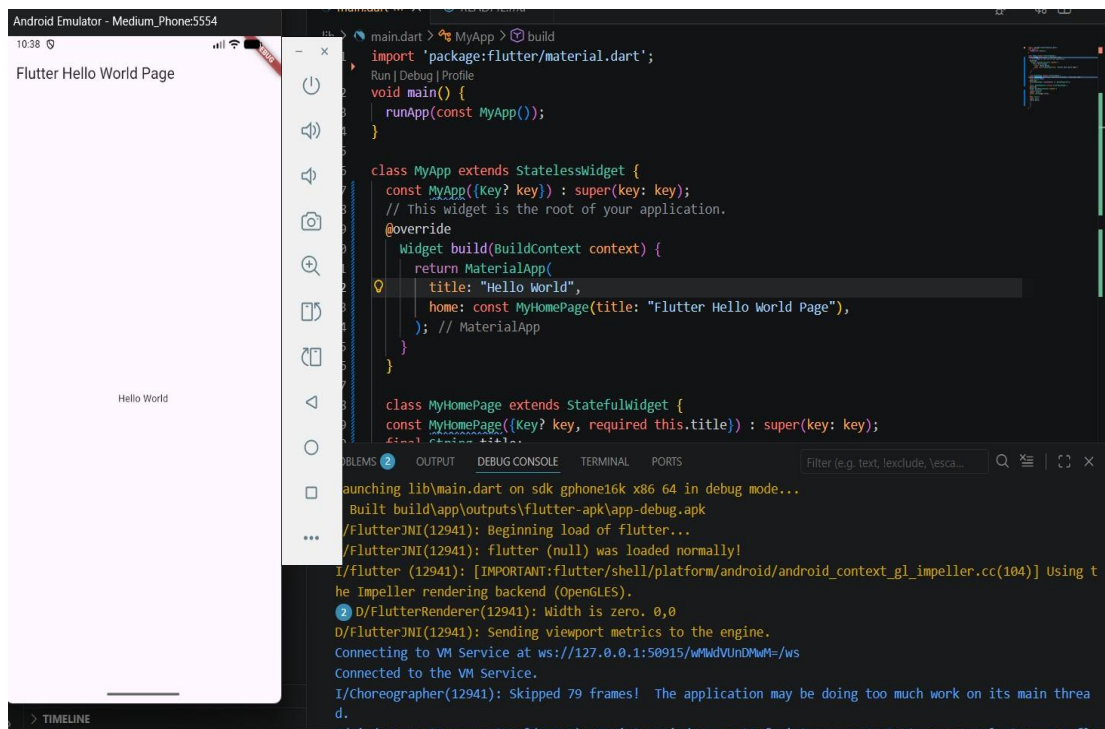
```

}

class MyHomePage extends StatefulWidget {
  const MyHomePage({Key? key, required this.title}) : super(key:
key);
  final String title;
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: Text(
          'Hello World',
        ),
      ),
    );
  }
}

```



MODUL 3

Pengenalan Dart

DASAR TEORI

3. Dart

Bahasa pemrograman Dart memiliki gaya penulisan sintaks yang sangat menyerupai bahasa pemrograman C maupun Java. Salah satu aturan mutlak dalam penulisannya adalah setiap baris perintah wajib diakhiri dengan tanda titik koma (;).

3.1. Variable

Dart menyediakan beberapa tipe data primitif standar, yaitu Integer, Double, String, dan Boolean. Dalam mendeklarasikan variabel, programmer bisa menggunakan tiga pendekatan, yaitu menggunakan kata kunci var, menuliskan tipe datanya secara eksplisit (type annotation), atau mendeklarasikan banyak variabel sekaligus dalam satu baris (multiple variable).

```
// var
var <variable_name>;
var <name> = <expression>;

// type annotation
<type> <variable_name>;
<type> <name> = <expression>;
// multiple variable
<type> <var1,var2...varN>;
```

3.2. Statement Control

Mengatur alur logika dan pengambilan keputusan, Dart mendukung beberapa jenis struktur pengkondisian:

3.2.1. IF Statement

```
if(condition){
    // statements
}
```

Digunakan untuk mengeksekusi sebuah blok kode (*statements*) hanya jika sebuah kondisi bernilai benar.

3.2.2. IF ELSE Statement

```
// IF ELSE STATEMENT
if(condition){
    // statements
} else {
    // statements
}
```

Digunakan untuk memberikan jalan alternatif, jika kondisi pertama bernilai salah, maka blok kode di dalam 'else' yang akan dieksekusi.

3.2.3. *IF ELSE IF Statement*

```
// IF ELSE IF STATEMENT
if(condition1) {
    // statement(s) }
else if(condition2){
    // statement(s) }..
else if(conditionN){
    // statement(s) }
else {
    // statement(s) }
```

Digunakan ketika terdapat lebih dari dua kemungkinan kondisi yang harus dievaluasi secara berurutan.

3.2.4. *SWITCH CASE Statement*

```
// SWITCH CASE
switch(expression){
case value1: {
    // statements }
    break;
case value2: {
    // statements }
    break;
default: {
    // statements }
    break;
}
```

Digunakan sebagai alternatif pengkondisian untuk mengevaluasi satu variabel (ekspresi) terhadap banyak kemungkinan nilai (*cases*).

3.3. *Looping*

Secara umum, Dart menyediakan dua cara untuk melakukan perulangan kode:

3.3.1. *For Loops*

Metode ini sangat *ideal* digunakan apabila jumlah iterasi perulangannya sudah diketahui secara pasti sebelumnya.

```
for (initial_count_value; termination-condition; step) {
    //statements
}
```

3.3.2. *While Loops*

Metode ini cocok digunakan ketika batas atau kapan perulangan tersebut akan berhenti belum diketahui secara pasti, dan akan terus berjalan selama kondisinya masih terpenuhi.

```
while (expression) {
    // Statement(s) to be executed if expression is true
}
```


3.4. *List*

Berbeda dengan beberapa bahasa pemrograman yang menyebutnya *Array*, Dart menggunakan istilah *List* untuk menyimpan kumpulan banyak data di dalam satu variabel.

3.4.1. *Fixed Length List*

Jenis *List* yang panjang indeksnya telah dikunci (tetap) sejak awal, sehingga jumlah datanya tidak bisa ditambah lagi setelah dideklarasikan.

```
// Mendeklarasikan list
var list_name = new List(initial_size);

// Menginisialisasikan list
list_name[index] = value;

// Contohnya
var newList = new List(3);
PEMROGRAMAN PERANGKAT BERGERAK 40
newList[0] = 12;
newList[1] = 13;
newList[2] = 11;
```

3.4.2. *Growable List*

Jenis *List* yang bersifat dinamis, tipe ini sangat berguna apabila jumlah objek data tidak menentu dan akan terus bertambah seiring berjalannya program.

```
// Mendeklarasikan list
var list_name = new List();

// Menginisialisasikan list
list_name[index] = value;

// Contohnya
var newList = new List(3);
newList[0] = 12;
newList[1] = 13;
newList[2] = 11;
```

3.5. Fungsi

Dalam pemrograman berorientasi objek, pembuatan fungsi atau prosedur sangat *vital* untuk menjaga kualitas kode, menerapkan prinsip *separation of concern* (setiap bagian punya tugas spesifik), serta meminimalisir penulisan kode berulang (*boilerplate code*).

3.5.1. Mendefinisikan Fungsi

```
void function_name() {  
    //statements  
}
```

Langkah awal untuk menyusun sebuah blok kode yang berisi instruksi-instruksi spesifik agar nantinya bisa dipanggil kapan saja.

3.5.2. Memanggil Fungsi

```
void main() {  
    print(factorial(6));  
}
```

Cara untuk menjalankan instruksi-instruksi yang telah dibuat di dalam fungsi tersebut pada bagian utama program (seperti di dalam *void main()*).

3.5.3. Mengembalikan Nilai

```
factorial(number) {  
    if (number <= 0) {  
        // termination case  
        return 1;  
    } else {  
        return (number * factorial(number - 1));  
        // function invokes itself  
    }  
}
```

Fungsi dapat menghasilkan sebuah nilai keluaran untuk dikembalikan ke baris yang memanggilnya dengan menggunakan perintah *return*.

3.5.4. Menambahkan Parameter

```
factorial(number) {  
    if (number <= 0) {  
        // termination case  
        return 1;  
    } else {  
        return (number * factorial(number - 1));  
        // function invokes itself  
    }  
}
```

Karena sebuah fungsi memiliki ruang lingkup (*scope*) yang terbatas, fungsi sering kali membutuhkan input dari luar agar dapat memproses data sesuai kebutuhan program.

Link GitHub: <https://github.com/valishaatthalia/2311102160-Valisha-Webpro.git>